DM872 – Mathematical Optimization at Work
Assignment 2, Spring 2026

Marco Chiarandini and Konstatin Pavlikov May 10, 2026

Preface

§1. This is the second of the two assignments that will be graded and will determine your final assessment in DM872.

§2. **Submission Deadline: Wednesday, May 31, 2026, at 23:59.**

§3. **This assignment can be carried out in groups of two or individually**. You are not allowed to collaborate by any means with other persons than the one in your group.

§4. To complete the assignment you have to submit in ItsLearning a '.tgz' or '.zip' archive named 'asg2.tgz' or 'asg2.zip' that decompress in the same structure as the 'asg2' directory in this repository:

```
asg2/
|-  README.md
|-  data
|- src
|- tex
    |- main.pdf
```

- `data` contains the problem instances
- `tex` contains your report in pdf format as concise as possible with the answers to the tasks.
- `src` must contain the Python code for the calculations you carried out. However, *additionally* consider sharing a gitlab.sdu.dk repository.

§5. Questions and answers about this assignment cannot be addressed to the teachers but must be posted as issues in the github page Issues. Start the title with `[asg2]`. In this way, the communication will be visible to the other type members as well.

§6. Changes to this document after publication on May 9 will be emphasized by a different color.

§7. You must declare the use of generative AI (GAI) using the GAI declaration form developed at the Faculty of Science. If no declaration is attached, and it is not otherwise clearly stated that SDU's declaration rules have been followed, and the assignment shows signs of GAI use or there is suspicion of such use, the case must be reported as cheating in exams in

accordance with SDU's rules on the use of GAI. You can read more about SDU's rules on the use of generative AI at https://mitsdu.dk/en/aiatsdu.

In this course, you must restrict the use of AI tools to questions about the programming language python and about writing style of the report. You must not get aid on the technical part of the tasks (ie, the modelling and the decomposition aspects). If you use AI, the teachers will be allowed to take this in consideration and to lower consequently the final grade if they suspect that the learning goals have not been achieved.

§8. You can write your answers in Danish or in English.

Introduction

A cargo company uses a heterogenous fleet of vessels to carry cargos measured in number of containers. Each vessel type t has its own specific capacity u_t, expressed in units of cargo, draft, and design speed v_t. Each vessel provides a *service* (or route) starting at an origin port and ending at a destination port, visiting transhipment ports in between. All together, this establishes the *service network* of the cargo company, consisting of a set of unique ports connected by the shipping services offered by the company. All services are periodic and may take several weeks to rotate. However, here we consider the single-voyage allocation: one vessel makes one trip on each route, and demand is treated as a one-time quantity rather than a weekly flow.

The company has a list of demands $D = \{(o_k, d_k, q_k, r_k) \mid k = 1, \ldots, s\}$ to be transported, where each demand is specified by its origin o_k and destination d_k ports, by the quantity q_k in number of containers it consists of and by the revenue r_k it creates, that is, the total income generated from sales before any expenses are deducted.

Transit from a port to another has an arc dependent cost per unit of demand, c_{ij}^k. Loading and unloading of demands at ports has a cost per unit of demand, e_i. Further, transshipment (transfer between vessels) at a port has a cost per unit of demand, f_i. For each unit of demand, the loading cost is paid at the origin port, the unloading port at the destination port and the transhipment cost at all other ports visited different from the origin and destination ports.

At strategy level (long term horizon), the company has to decide the *service network* and the *demands to be transported*, maximizing the profit, that is, what remains of the revenue once costs are subtracted, while respecting the capacity constraints of the vessels.

In this assignment: First, you will consider a simplified version of the problem, **RP**, where the service network will be relaxed and the company has only to decide *demands to be transported*, that is, for each demand the arcs of the network (see Figure 1) that should be used to transport it, and how much demand to load and unload at each port.

Then, if you are able to solve the first problem, you will be asked to consider the full problem, **FP**, where the company has to decide both the *service network and the demands to be transported*.

Remarks

FP & RP You will not be able to route all demands but maximizing profits you will leave out those whose expenses are more than the revenue created.

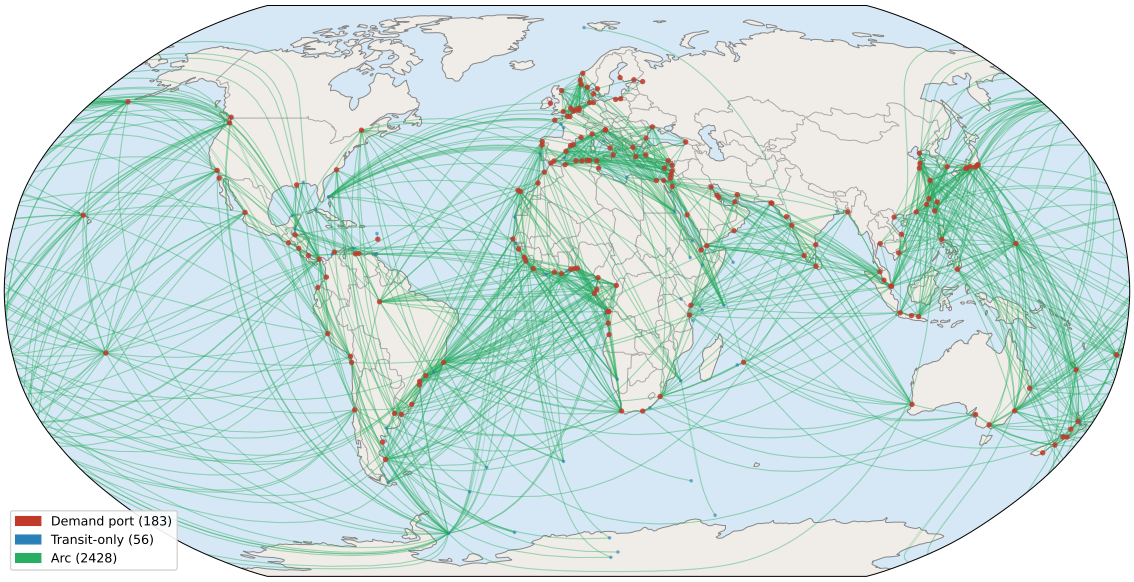

Figure 1: The arc network from the instances. Some ports do not have coordinates and are not depicted. Arcs are represented as edges for which transit in both directions is allowed.

FP & RP It is possible to accept a lower number of containers than the value of the original demand if that is overall more profitable.

FP & RP In practice, because the number of containers transported in a global shipping network is huge, rounding down the fractional part of demands may be considered insignificant. Hence, the demand transported can be a fractional value.

FP A service network is a set of services (or routes): each service is made by a vessel that departs from a port, visits a sequence of ports, and returns to the origin. The arcs on that service form a cycle, and the same vessel traverses all of them.

This creates two kinds of dependency:

- Capacity coupling across arcs: if a vessel serves ports $A \to B \to C \to A$, the cargo loaded at A and bound for C occupies space on both arc $A \to B$ and arc $B \to C$.
- Fleet balance: the number of vessels on a service is determined by the round-trip time divided by the service frequency (e.g. weekly). A vessel cannot be simultaneously on arc $A \to B$ and arc $D \to E$ of a different service — the fleet is shared and finite.

In the absence of predefined services, the problem consists in deciding how many vessels of each type to place on each arc, which then determines the capacity available to the cycles. The vessel deployment cost becomes a fixed cost in the objective.

RP A reasonable relaxation of FP is to decouple the arcs and assume that each arc can be served by one (or multiple) separate vessel, which is not realistic but allows to solve the problem faster. On each arc, for each demand the vessel types chosen are the ones that have the largest capacity and the fastest speed among the feasible ones for that arc. If the

feasible vessel type with the largest capacity differ from the one with the fastest speed, it is allowed to assume another type exists with both peak characteristics.

In this setting, the example above the cycle $A \to B \to C \to A$ is treated as three independent arcs $A \to B$, $B \to C$, and $C \to A$, each with its own vessel and capacity, so it is possible to fill all arcs to capacity with different cargo. The assumption of three different vessels per arc is not just a simplification of the optimisation, it's also physically unrealistic for most arcs, since a single vessel can only be in one place at a time on a multi-leg route.

Once service routes and fleet assignments are fixed (i.e., the network design subproblem is already solved) and **RP** uses those fixed arc capacities and speeds, then it provides the exact solution but as a standalone model it overstates the available capacity by decoupling the arcs.

Your Tasks

1. Develop a mathematical programming model to solve **RP**. Define the decision variables, constraints and the objective function. Implement the introduced mathematical programming model and solve it for at least two instances (see also more details below):

   - "Baltic" with 456 ports, 5156 arcs and 22 demands
   - "WorldLarge" with 456, 5156 arcs and 9,615 demands

   Report the solutions you obtain for these two problem instances with a time limit of 1000 seconds. If you are not able to obtain a solution for either of the problem instances,

   - argue whether one among the decomposition methods considered in class can be helpful to address the problem
   - implement the identified decomposition method(s) and report solutions to the problem instances using the decomposition method(s).

2. Develop a mathematical programming model to solve **FP**. Implement the model and solve it for the smallest instance.

Data Instances

The dataset was assembled from real-world data and cost figures were simply not available for ports that are rarely or never used as primary loading/unloading points. For these ports (i.e., when NULL costs) the fall back is to set transhipment costs to 0 (i.e., cost_transship = 0), those ports can still be used for routing, the transshipment cost is just not penalised, which is a slight underestimation. Given that no demand is ever loaded or unloaded at those ports, their cost_load values never come into play either.

Demand load is measured in ffe (Forty-foot Equivalent) unit, which corresponds to one 40-foot shipping container (equivalent to two TEUs, Twenty-foot Equivalents). So ffe for a demand is the number of 40-foot containers that need to be shipped from the origin port to the destination port.

The structure of every `distilled_<instance>.json` file is as follows:

instance string, the instance name (e.g. "Baltic").

nodes list of UNLocode strings (e.g. "DEBRV"). These are all ports that appear as endpoints of at least one arc in the distance network. There are always 456 of them, shared across all instances.

arcs dict keyed by "ORIGIN,DEST" (two UNLocodes joined by a comma). Each value has:

- **distance** nautical miles between the two ports
- **feasible_vessels** list of vessel type names whose draft does not exceed the shallowest of the two ports

vessels dict keyed by vessel type name (e.g. "Feeder_450"). Only the types present in the instance's fleet file are included. Each value has:

- **capacity** maximum load in FFE
- **design_speed** cruising speed in knots
- **bunker_per_day** fuel consumption in metric tons/day at design speed
- **tc_rate_daily** daily time-charter rate in USD (fixed operating cost)

ports dict keyed by UNLocode (435 ports with cost data). Each value has:

- **cost_load** USD per FFE to load or unload at this port (null for 145 ports with missing data)
- **cost_transship** USD per FFE to transship (transfer between vessels) at this port (null for the same ports)

demands dict keyed by "ORIGIN,DEST". Only routable pairs are included (both ports present in the network). Each value has:

- **ffe** number of 40-foot containers to be shipped
- **revenue** USD per FFE earned if the cargo is transported

The transit cost per unit of demand, c_{ij}^k, is calculated as follows. For a vessel of type t serving the arc ij one first calculates the number of days needed to cover the service (ndays = $\text{distance}_{ij}/\text{design\_speed} \times 24$). Then,

$$c_{ij}^k = (\text{bunker\_per\_day}_t \times \text{BUNKER\_PRICE} + \text{tc\_rate\_daily}_t) \times \text{ndays}$$

The BUNKER_PRICE is the price per metric ton of bunker fuel and is fixed to 500 USD.

The network (nodes/arcs) is shared across all instances and it is represented in Figure 1. The number of nodes differ because some ports do not have geographical coordinates in the data. Instances differ only in fleet and demand. Statistics of the instances are reported in Figure 2. "Routable" excludes demands whose origin or destination port is absent from the distance network.

In the directory where this document is store you find the instance files and a starting script carrying out the cost calculations described above.

Instance	Nodes	Arcs	Vessel types	Demands	Routable	Total FFE	Potential revenue
Baltic	456	5156	2	22	22	4,904	4,054,660
Mediterranean	456	5156	3	365	296	5,950	4,340,170
WAF	456	5156	2	37	37	8,541	15,000,250
Pacific	456	5156	4	722	587	35,642	36,935,780
EuropeAsia	456	5156	6	4,000	3,705	67,588	118,239,640
WorldSmall	456	5156	6	1,764	1,229	93,430	172,318,586
WorldSmall_Fixed_Sep	456	5156	6	1,764	1,229	100,187	191,285,800
WorldLarge	456	5156	6	9,615	8,061	105,405	208,186,280

Figure 2: Instance statistics